Swiggy SQL Data Analysis

Customer, Restaurant, Order and Delivery Performance Analysis



Prepared by: *Shivji*
Tools Used: *MySQL / SQL Joins / Aggregations / Subqueries*

Project Overview

Objective:

Analyse Swiggy’s SQL database to understand customer activity, restaurant performance, revenue and delivery operations.

Analysis Focus:

- Customer ordering behaviour
- Restaurant ratings and revenue
- Recent order activity
- Delivery-partner performance
- Customer and restaurant relationships

Dataset Overview

The database contains interconnected tables covering:

- **Customers:** Customer details and cities
- **Restaurants:** Restaurant location and ratings
- **Orders:** Order amount, date and status
- **Delivery Partners:** Delivery-partner details
- **Order Delivery:** Partner-to-order assignment
- **Delivery Updates:** Delivery status tracking

Key Point:
Primary and foreign keys were used to combine related information across multiple tables.

SQL Techniques Used

- **WHERE** for filtering records
- **JOIN** for combining related tables
- **LEFT JOIN** for retaining unmatched records
- **GROUP BY** for category-level analysis
- **COUNT(), SUM() and AVG()** for calculations
- **HAVING** for filtering aggregated results
- **EXISTS** and **NOT EXISTS** for customer activity
- Self-joins for comparing different customers



Delhi Customer Analysis

Display all customers who live in 'Delhi'

SQLQUERY

```
SELECT
    customer_id,
    name,
    email,
    phone_number,
    city
FROM Customers
WHERE city = 'Delhi';
```

OUTPUT

customer_id	name	email	phone_number	city
2	Rohini Verma	rohini.verma@yahoo.com	9823456789	Delhi
5	Manish Kumar	NULL	9834567890	Delhi
18	Sonali Mishra	NULL	9878345678	Delhi

Query Logic:

The WHERE clause filters the Customers table and returns only customers whose city is Delhi.

Key Insight:

The query identified **3 Delhi-based customers**. However, **2 of the 3 email records are missing**, which may limit email-based campaigns. Phone numbers can be used for local offers after contact validation.

Mumbai Restaurant Rating Analysis

Find the average rating of all restaurants in 'Mumbai'

SQLQUERY

```
SELECT
    ROUND(AVG(rating), 2) AS average_rating
FROM Restaurants
WHERE city = 'Mumbai';
```

OUTPUT

average_rating
4.30

Query logic

The WHERE clause selects only restaurants located in Mumbai. The AVG() function calculates their overall average rating, while ROUND() displays the result to two decimal places.

Key insight

Mumbai restaurants have an average rating of **4.30**, indicating strong overall customer satisfaction. Swiggy can promote highly rated Mumbai restaurants while reviewing lower-rated restaurants individually.

Active and Inactive Customers

Which customers placed at least one order?
Which customers never placed an order?

SQLQUERY

```
SELECT c.customer_id,
       c.name,
       c.city
FROM Customers c
WHERE EXISTS (
    SELECT 1
    FROM Orders o
    WHERE o.customer_id = c.customer_id
);
```

OUTPUT

customer_id	name	city
1	Amit Sharma	Mumbai
2	Rohini Verma	Delhi
3	Rajesh Gupta	Mumbai
4	Sneha Mehta	Pune
5	Manish Kumar	Delhi
6	Priya Singh	Kolkata

Logic:
EXISTS checks whether at least one order is connected to each customer.

Key insight:
Identifies active customers who have already used Swiggy and can be targeted for repeat-order offers.

Inactive Customers

Display all customers who have never placed an order

SQLQUERY

```
SELECT c.customer_id,
       c.name,
       c.city
FROM Customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM Orders o
    WHERE o.customer_id = c.customer_id
);
```

OUTPUT

customer_id	name	city
24	Sonal Kaur	Amritsar
25	Vivek Malhotra	Thane
26	Divya Iyer	Bangalore
27	Rakesh Yadav	Varanasi
28	Mona Sharma	Ranchi
29	Sudha Pillai	Kozhikode

Logic:
NOT EXISTS returns customers who do not have any matching record in the Orders table.

Key insight:
These registered but inactive customers can be targeted with first-order discounts or reminder notifications.

Customer Order Frequency

Display the total number of orders placed by each customer

SQLQUERY

```
SELECT c.customer_id,
       c.name,
       COUNT(o.order_id) AS total_orders
FROM Customers c
LEFT JOIN Orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_orders DESC;
```

OUTPUT

customer_id	name	total_orders
5	Manish Kumar	4
2	Rohini Verma	3
3	Rajesh Gupta	3
6	Priya Singh	3
7	Vikas Reddy	3

Logic:
LEFT JOIN includes every customer, including customers with zero orders. COUNT() calculates their number of orders.

Key insight:
Shows the most active customers and also highlights customers with low or no ordering activity.

Customer Order Frequency

Find the number of orders placed by each customer in Mumbai

SQLQUERY

```
SELECT c.customer_id,
       c.name,
       COUNT(o.order_id) AS total_orders
FROM Customers c
LEFT JOIN Orders o
    ON c.customer_id = o.customer_id
WHERE c.city = 'Mumbai'
GROUP BY c.customer_id, c.name
ORDER BY total_orders DESC;
```

OUTPUT

customer_id	name	total_orders
3	Rajesh Gupta	3
1	Amit Sharma	2
19	Arjun Desai	2
23	Ravi Singh	2

Logic:
Filters customers living in Mumbai and then counts their orders. LEFT JOIN also keeps Mumbai customers who placed zero orders.

Key insight:
Helps compare ordering activity among Mumbai customers and identify frequent and inactive users.

Customer Order Frequency

Which customers ordered on exactly three different days?

SQLQUERY

```
SELECT c.customer_id,
       c.name,
       COUNT(DISTINCT DATE(o.order_date)) AS different_order_days
FROM Customers c
JOIN Orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
HAVING COUNT(DISTINCT DATE(o.order_date)) = 3;
```

OUTPUT

customer_id	name	different_order_days
2	Rohini Verma	3
6	Priya Singh	3
8	Anjali Patel	3
14	Nidhi Saxena	3
15	Ashok Kumar	3
18	Sonali Mishra	3

Logic:

DATE() removes the time portion. COUNT(DISTINCT) counts unique ordering dates, and HAVING keeps customers with exactly three dates.

Key insight:

Identifies repeat customers who ordered across three separate days, rather than customers who placed multiple orders on the same day.

Restaurant Revenue Analysis

Find the total revenue generated by each restaurant

SQLQUERY

```
SELECT r.restaurant_id,
       r.name AS restaurant_name,
       COALESCE(
           SUM(
               CASE
                   WHEN o.status = 'Completed'
                   THEN o.total_amount
                   ELSE 0
               END
           ), 0
       ) AS total_revenue
FROM Restaurants r
LEFT JOIN Orders o
    ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.name
ORDER BY total_revenue DESC;
```

OUTPUT

restaurant_id	restaurant_name	total_revenue
3	Biryani House	5300.00
19	Awadhi Zaika	4150.00
10	Andhra Spice	4050.00
13	South Treat	2950.00
9	Gujarat Express	2550.00
7	Coastal Delight	2100.00

Logic:

Restaurants are joined with orders, and SUM() calculates revenue from completed orders. COALESCE() displays zero for restaurants without revenue.

Key Insight:

The analysis identifies high-revenue restaurants and highlights underperforming partners that may require promotional or operational support.

Top-Rated Restaurants

Find the five restaurants with the highest average rating

SQLQUERY

```
SELECT restaurant_id,
       name AS restaurant_name,
       city,
       ROUND(AVG(rating), 2) AS average_rating
FROM Restaurants
WHERE rating IS NOT NULL
GROUP BY restaurant_id, name, city
ORDER BY average_rating DESC, restaurant_name
LIMIT 5;
```

OUTPUT

restaurant_id	restaurant_name	city	average_rating
3	Biryani House	Hyderabad	4.80
22	Paradise Biryani	Hyderabad	4.80
30	Lucknowi Nawabi	Lucknow	4.70
6	Royal Biryani	Kolkata	4.70
19	Awadhi Zaika	Lucknow	4.60

Logic:
Restaurants are sorted by average rating in descending order, and LIMIT 5 returns the top performers.

Key Insight:
Highly rated restaurants are suitable for featured listings, premium placement and customer recommendation campaigns.

Recent Order Activity

Display orders placed during the latest 30-day period

SQLQUERY

```
SELECT o.order_id,
       c.name AS customer_name,
       r.name AS restaurant_name,
       o.order_date,
       o.total_amount,
       o.status
FROM Orders o
JOIN Customers c
    ON o.customer_id = c.customer_id
JOIN Restaurants r
    ON o.restaurant_id = r.restaurant_id
WHERE o.order_date >= DATE_SUB(
          (SELECT MAX(order_date) FROM
Orders),
          INTERVAL 30 DAY
      )
  AND o.order_date <= (
          SELECT MAX(order_date)
          FROM Orders
      )
ORDER BY o.order_date DESC;
```

OUTPUT

order_id	customer_name	restaurant_name	order_date	total_amount	status
54	Ashok Kumar	Awadhi Zaika	2024-08-15 00:00:00	1200.00	Completed
46	Arjun Desai	Awadhi Zaika	2024-08-15 00:00:00	1150.00	Completed
33	Priya Singh	Spice of India	2024-08-14 00:00:00	650.00	Completed
52	Deepa Rao	Gujarat Express	2024-08-14 00:00:00	950.00	Completed
32	Kavita Deshmukh	Awadhi Zaika	2024-08-13 00:00:00	1000.00	Completed
51	Sonali Mishra	Andhra Spice	2024-08-13 00:00:00	1100.00	Completed

Logic:

The maximum order date in the historical dataset is used as the reference, and orders from the previous 30 days are selected.

Key Insight:

Recent-order analysis helps monitor current demand, order value, restaurant activity and order-status performance.

Delivery-Partner Performance

Which partners completed more than one delivery?
Which partner served the most different customers?

SQLQUERY

```
SELECT dp.partner_id,
       dp.name AS partner_name,
       COUNT(DISTINCT od.order_id) AS completed_deliveries
FROM DeliveryPartners dp
JOIN OrderDelivery od
    ON dp.partner_id = od.partner_id
JOIN DeliveryUpdates du
    ON od.order_id = du.order_id
WHERE du.status = 'Delivered'
GROUP BY dp.partner_id, dp.name
HAVING COUNT(DISTINCT od.order_id) > 1
ORDER BY completed_deliveries DESC;
```

OUTPUT

partner_id	partner_name	completed_deliveries
4	Suresh Reddy	3
2	Ravi Kumar	2
5	Anita Desai	2
12	Reena Rao	2

SQLQUERY

```sql
SELECT dp.partner_id,
       dp.name AS partner_name,
       COUNT(DISTINCT o.customer_id) AS different_customers
FROM DeliveryPartners dp
JOIN OrderDelivery od
    ON dp.partner_id = od.partner_id
JOIN Orders o
    ON od.order_id = o.order_id
GROUP BY dp.partner_id, dp.name
ORDER BY different_customers DESC, dp.partner_id
LIMIT 1;
```

OUTPUT

partner_id	partner_name	different_customers
4	Suresh Reddy	6

Logic:

Delivery assignments are joined with delivery updates and orders. Completed deliveries and unique customers are then counted for each partner.

Key Insight:

The results identify experienced and highly active delivery partners who have strong delivery coverage and customer reach.

Customer–Restaurant Pattern

Identify same-city customers who ordered from the same restaurant on different dates.

Logic:

- The Customers table is joined with itself to create customer pairs.
- c1.city = c2.city ensures both customers live in the same city.
- The restaurant IDs must match.
- The order dates must be different.
- c1.customer_id < c2.customer_id prevents duplicate reversed pairs.

Key insight:

Reveals restaurants that attract multiple customers from the same city on different days, which can support city-level restaurant promotions.

SQLQUERY

```
SELECT DISTINCT
       c1.name AS customer_1,
       c2.name AS customer_2,
       c1.city,
       r.name AS restaurant_name,
       DATE(o1.order_date) AS
customer_1_order_date,
       DATE(o2.order_date) AS
customer_2_order_date
FROM Customers c1
JOIN Orders o1
    ON c1.customer_id = o1.customer_id
JOIN Customers c2
    ON c1.city = c2.city
   AND c1.customer_id < c2.customer_id
JOIN Orders o2
    ON c2.customer_id = o2.customer_id
   AND o1.restaurant_id = o2.restaurant_id
   AND DATE(o1.order_date) <>
DATE(o2.order_date)
JOIN Restaurants r
    ON o1.restaurant_id = r.restaurant_id
ORDER BY c1.city,
         restaurant_name,
         customer_1,
         customer_2;
```



OUTPUT

customer_1	customer_2	city	restaurant_name	customer_1_order_date	customer_2_order_date
Manish Kumar	Sonali Mishra	Delhi	Biryani House	2024-08-04	2024-08-05
Manish Kumar	Sonali Mishra	Delhi	Biryani House	2024-08-07	2024-08-05
Arjun Desai	Ravi Singh	Mumbai	Veggie Delight	2024-08-03	2024-08-09

Thank You

Dataset Link :-
https://drive.google.com/file/d/1S32wPjwNUlhi2G5xiW3_wZvGVKipJq-b/view?usp=sharing